הכנה לראיון: שתי התשובות המרכזיות

פרויקט AI Stock Research Dashboard · אופיר ערן

חלק א' — הפתיח: "ספר לי על הפרויקט"

זו השאלה הוודאית ביותר בכל ראיון. המבנה הוא ארבעה חלקים בסדר קבוע: מי אני, מה המערכת עושה, איך היא בנויה, ומה בניתי סביבה. הטקסט למטה הוא לשינון, כשישים שניות.

1. מי אני (כ-10 שניות)

> אני אופיר, מתחיל שנה ג' למדעי המחשב באקדמית תל אביב. בזמן החלמה מפציעה רציתי לבנות משהו שלם בעצמי, ומניות תמיד עניינו אותי, אז בניתי אפליקציית מחקר מניות מקצה לקצה.

2. מה המערכת עושה (כ-15 שניות)

> המשתמש מזין סימול של מניה. המערכת מושכת מחירים והיסטוריה של חודש, משווה את תשואת המניה לתשואת תעודת הסל של הסקטור שלה, כלומר בודקת אם היא הכתה את הענף, ומודל שפה מייצר ניתוח קצר: ציון סנטימנט, טיעוני קנייה ומכירה, והשוואה לסקטור.

3. איך היא בנויה (כ-20 שניות)

> ממשק ב-Streamlit שמדבר ב-HTTP עם שרת FastAPI. השרת בודק קודם מטמון ב-PostgreSQL, ורק אם אין שם תוצאה טרייה הוא מושך נתונים חיים ופונה למודל. יש שכבת ניתוב שמאפשרת להחליף בין Claude ל-OpenAI דרך משתנה סביבה, ואם אין מפתח כלל, המערכת מחזירה ניתוח מדומה כדי שהיא תרוץ מקצה לקצה גם בלי הרשאות.

4. מה בניתי סביב זה (כ-15 שניות)

> ארזתי את שני השירותים בקונטיינרים והרצתי אותם יחד עם Docker Compose, כתבתי תשע עשרה בדיקות אוטומטיות שרצות ב-GitHub Actions בכל דחיפת קוד, וכתבתי תשתית ב-Terraform ל-AWS ומניפסטים ל-Kubernetes. שניהם מאומתים אוטומטית באותו תהליך.

שלושה משפטים שאסור לומר:

- **"פרסתי בענן"** — לא הרצת terraform apply. אמור: "כתבתי ואימתתי את התשתית; לא הרצתי אותה בפועל כי RDS ו-ALB עולים כסף חודשי."
- **"הרצתי קוברנטיס"** — אם לא הרמת אשכול, אמור: "כתבתי מניפסטים ל-Kubernetes".
- **"אני לא זוכר הרבה מהקורס"** — אל תתנדב לספר מה שכחת. תאר מה למדת ועצור.

דיוק שכבות: ה-backend שלך הוא FastAPI. Render היא פלטפורמת האירוח שבה הוא רץ. אלה שתי שכבות שונות, אל תערבב ביניהן.

חלק ב' — שישה מונחי יסוד

לכל מונח: הגדרה מדויקת במשפט אחד, ומיד אחריה הקישור לקוד שלך. החיבור בין התיאוריה לפרויקט הוא מה שמבדיל תשובה של סטודנט מתשובה של מהנדס.

מונח	ההגדרה	איפה זה אצלך
סיבוכיות חיפוש dict מול list	חיפוש איבר ברשימה הוא O(n), כי עוברים איבר אחר איבר. במילון זה O(1) בממוצע, כי המילון ממומש כטבלת גיבוב: מחשבים hash של המפתח וקופצים ישירות אליו. לכן המפתח חייב להיות בלתי משתנה, אחרת ה-hash יפסיק להצביע למקום הנכון.	המילון SECTOR_ETF_MAP ממפה סקטור לתעודת סל בגישה מיידית. השורה get(sector, "SPY"). מחזירה SPY, מדד השוק הרחב, כברירת מחדל כשהסקטור לא במפה.
PRIMARY KEY	מעבר לייחודיות, מפתח ראשי יוצר אוטומטית **אינדקס**, ובפוסטגרס זהו אינדקס מסוג עץ חיפוש מאוזן. בלי אינדקס השליפה סורקת את כל הטבלה, כלומר O(n); עם אינדקס היא לוגריתמית. הוא גם מרמז NOT NULL.	עמודת הסימול היא המפתח הראשי בטבלת המטמון, ולכן שליפת מניה נשארת מהירה גם כשהטבלה גדלה.
אידמפוטנטיות idempotent	פעולה שביצוע חוזר שלה אינו משנה את מצב המערכת מעבר לביצוע הראשון. GET אמור להיות בטוח ואידמפוטנטי.	חמש קריאות לאותו סימול יחזירו את אותו תוכן. יש תופעת לוואי אחת, כתיבה למטמון בקריאה הראשונה, אבל היא לא משנה מצב שהמשתמש רואה. מה שכן משתנה הוא השדה cached וזמן התגובה.
Mock בבדיקות	אובייקט מזויף שמחליף תלות אמיתית בזמן בדיקה. שלוש סיבות: דטרמיניזם, כי אני שולט בנתונים שחוזרים; מהירות, כי אין קריאות רשת; ויציבות, כי הבדיקה לא נשברת כשהשירות החיצוני נופל.	עשיתי mock ל-yfinance, ולכן יכולתי לבדוק מקרה קצה של היסטוריה ריקה, שאי אפשר לשחזר עם נתונים אמיתיים. כשיאהו החזיר תשובות בעייתיות בפרודקשן, הבדיקות נשארו ירוקות: הן בודקות את הקוד שלי, לא את הרשת.
Image מול Container	Image הוא תבנית בלתי משתנה, מעין מתכון, שנבנית פעם אחת. Container הוא	יש שתי תמונות נפרדות, אחת לשרת ואחת לממשק, וגם קבצי תלויות

	מופע רץ של אותה תבנית. מתמונה אחת אפשר להריץ קונטיינרים רבים.	נפרדים, כך שהממשק לא נושא את ספריות הנתונים הכבדות של השרת.
Cache stampede	כמה בקשות מגיעות בו זמנית לאותו מפתח שאינו במטמון, כולן מקבלות miss, וכולן מבצעות את העבודה היקרה במקביל. זו אינה סתם "עומס על המטמון".	שני משתמשים שמבקשים את אותה מניה באותה שנייה יגרמו לשתי קריאות למודל, כלומר עלות כפולה. בהיקף שלי זה זניח, והכתיבות לא מתנגשות כי הן על אותו מפתח ראשי. הפתרון אם היה נדרש: נעילה לפי סימול, כך שרק הבקשה הראשונה עובדת והשנייה ממתינה לתוצאה.

כלל אצבע לכל תשובה טכנית: הגדרה במשפט אחד, ואז משפט אחד שמחבר לקוד שלך. שני משפטים מספיקים, ומראים שאתה מבין ולא משנן.